Solution: Reverse Vowels

Problem Statement

Given a string `s`, reverse only all the vowels in the string and return it.

The vowels are `'a'`, `'e'`, `'i'`, `'o'`, and `'u'`, and they can appear in both lower and upper cases, more than once.

Example 1:

```
Input: s= "hello"
Output: "holle"
```

Example 2:

```
Input: s= "AEIOU"
Output: "UOIEA"
```

Example 3:

```
Input: s= "DesignGUrus"
Output: "DusUgnGires"
```

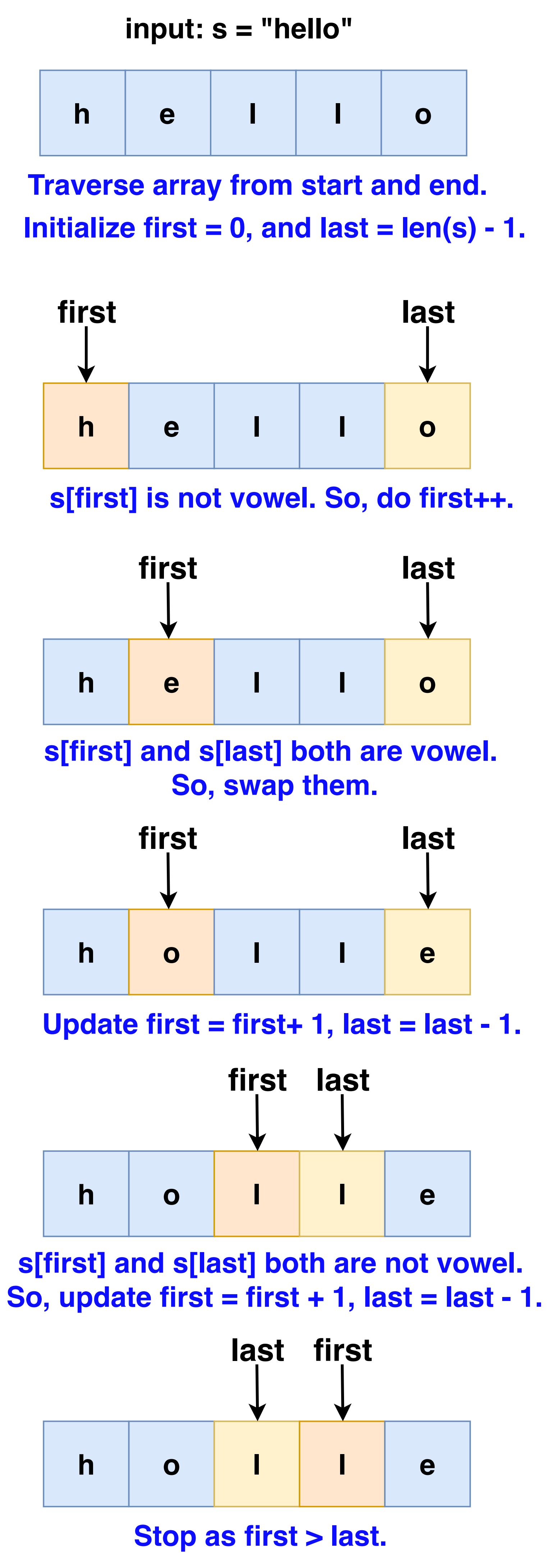
Solution

We need to reverse the vowels in a string, keeping the positions of the consonants and other characters intact. We can use the two-pointer technique to traverse the string from both ends simultaneously. Whenever a vowel is encountered at both ends, we will swap them. The process will continue until the two pointers meet in the middle.

Walk through of the algorithm

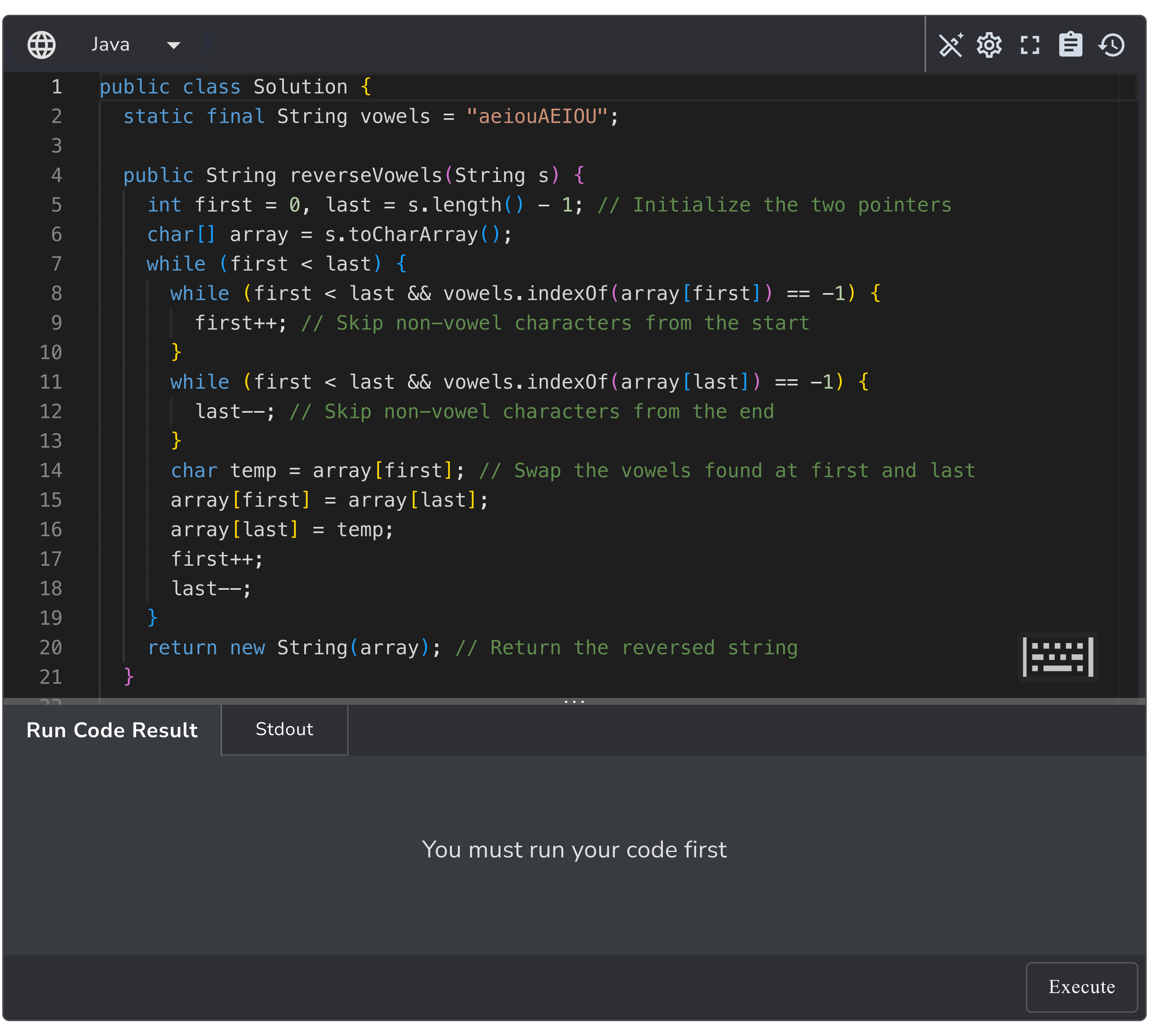
Now let's walk through the code:

1. We first create a static string "vowels" that contains all lowercase and uppercase vowels. This is used for checking whether a character in the input string is a vowel.
2. The `reverseVowels` method accepts a string 's' as an argument. The first and last pointers are initialized to the start and end of the string, respectively.
3. We convert the input string 's' to a character array 'array' to allow easy manipulation of individual characters.
4. We enter a while loop that continues while 'first' is less than 'last'.
5. Inside the while loop, we have two nested while loops:
   - The first nested while loop keeps incrementing the 'first' pointer until it points to a vowel or 'first' is no longer less than 'last'.
   - The second nested while loop keeps decrementing the 'last' pointer until it points to a vowel or 'first' is no longer less than 'last'.
6. Once we've found a vowel at both the 'first' and 'last' pointers, we swap these vowels.
7. After the swap, we increment the 'first' pointer and decrement the 'last' pointer, and continue to the next iteration of the outer while loop.
8. After exiting the while loop, we convert the character array back to a string and return it as the result.


input: s = "hello"
h e l l o
Traverse array from start and end.
Initialize first = 0, and last = len(s) - 1.
first
last
h e l l o
s[first] is not vowel. So, do first++.
first
last
h e l l o
s[first] and s[last] both are vowel.
So, swap them.
first
last
h o l l e
Update first = first+ 1, last = last - 1.
first
last
h o l l e
s[first] and s[last] both are not vowel.
So, update first = first + 1, last = last - 1.
last
first
h o l l e
Stop as first > last.

Code

Here is the code for this algorithm:



```java
public class Solution {
  static final String vowels = "aeiouAEIOU";

  public String reverseVowels(String s) {
    int first = 0, last = s.length() - 1; // Initialize the two pointers
    char[] array = s.toCharArray();
    while (first < last) {
      while (first < last && vowels.indexOf(array[first]) == -1) {
        first++; // Skip non-vowel characters from the start
      }
      while (first < last && vowels.indexOf(array[last]) == -1) {
        last--; // Skip non-vowel characters from the end
      }
      char temp = array[first]; // Swap the vowels found at first and last
      array[first] = array[last];
      array[last] = temp;
      first++;
      last--;
    }
    return new String(array); // Return the reversed string
  }
```

Time Complexity

1. `s.toCharArray() or list(s)`: Converting the string `s` to a character array is an `O(n)` operation, where `n` is the length of the string.
2. The two while loops inside the main loop also contribute to the time complexity:
   - The outer while loop (`while (first < last)`) runs at most `n/2` times because it stops when the two pointers meet in the middle.
   - The inner while loops (`while (first < last && vowels.indexOf(array[first]) == -1)` and its counterpart for `last`) will execute `indexOf` which is an `O(m)` operation where `m` is the length of the `vowels` string. However, since the length of the `vowels` string is a small constant (10 characters), it can be considered `O(1)` for each comparison in the context of Big O notation.
   - In the worst-case scenario, each character in `s` is compared once, so we can simplify the complexity of these loops to `O(n)` for the entire input string since all characters are processed at most once.

Hence, the combined time complexity for these operations is `O(n)`.

Space Complexity

1. `char[] array = s.toCharArray()`: Allocating a character array of the same size as the string `s` takes `O(n)` space.
2. The string `vowels` is a constant and doesn't scale with the input, so it doesn't add to the space complexity.
3. The pointers `first` and `last` use a constant amount of space.

The space complexity, therefore, is `O(n)` because of the character array that is created.

In conclusion, the `reverseVowels` method has a time complexity of `O(n)` and a space complexity of `O(n)`, where `n` is the length of the input string `s`.

← Previous
Reverse Vowels (Easy)

Next →
Valid Palindrome (Easy)

☐ Mark as Completed